

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Algoritmo Pérez

8 de septiembre de 2026

19:37

1. Introducción

El informe detalla el análisis experimental de cuatro algoritmos de ordenamiento unidimensional y dos algoritmos de multiplicación de matrices cuadradas. El objetivo fundamental es ir más allá de la notación asintótica teórica para evaluar el rendimiento práctico de estas implementaciones sobre hardware de un computador o dispositivo. Al someter los algoritmos a diferentes distribuciones y volúmenes de datos, se busca relacionar directamente su complejidad matemática con el tiempo de ejecución del algoritmo (programa), demostrando que factores como las constantes, la gestión de memoria y los peores casos teóricos tienen un impacto crítico en el poder de cómputo. Para poder entender esto a continuación se presenta el entorno experimental con el que se trabajó.

2. Entorno experimental

Las pruebas de rendimiento se ejecutaron en un entorno Linux Ubuntu 24.04.4 LTS mediante el compilador estándar de C++, El hardware corresponde a un HP Pavilion Gaming Laptop 15-cx0xxx, procesador Intel® Core™ i5-8300H $\times$ 8 y 16 GB de RAM. Los casos de prueba para el problema de ordenamiento abarcaron arreglos desde $N=10$ hasta 10.000.000 de elementos, evaluando distribuciones ascendentes, descendentes y aleatorias. Para la multiplicación de matrices, se procesaron estructuras desde la dimensión $N = 2^1$ hasta $N = 2^8$ bajo formatos densos, diagonales y dispersos.

3. Resultados y Análisis

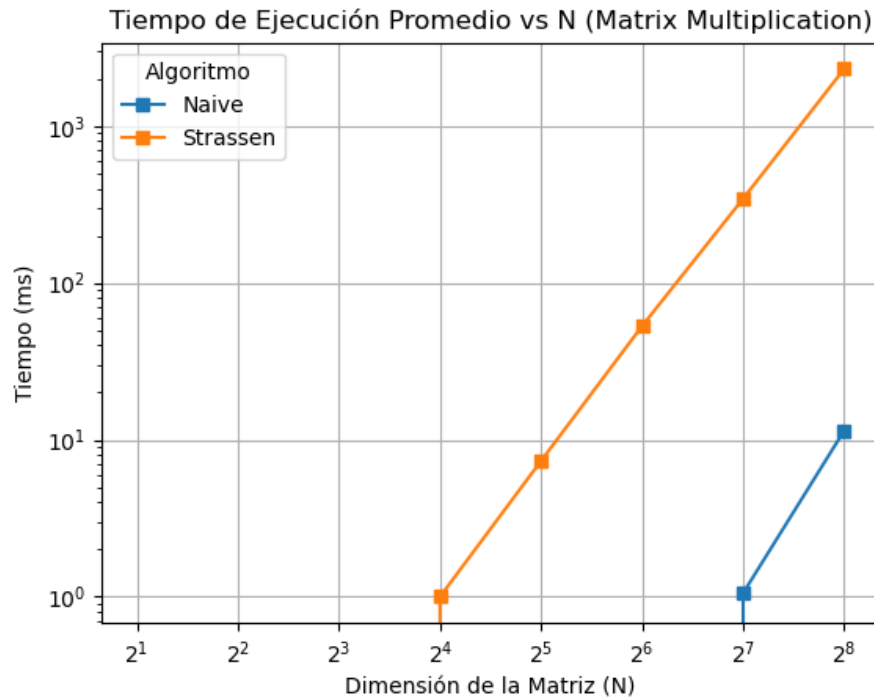


Figura 1: Tiempo de ejecución vs Dimensión de la matriz para los algoritmos Naive y Strassen.

3.1. Multiplicación de Matrices: El peso de las constantes

El gráfico generado desde `code/matrixmultiplication/data/plots/` revela que el algoritmo Naive supera empíricamente a Strassen para la dimensión máxima exigida de $N = 2^8$. Aunque teóricamente Strassen fue diseñado para reducir la cantidad de multiplicaciones escalares, su sobrecarga estructural y constante creación de submatrices saturan el procesador antes de que el volumen de datos logre compensar su costo operativo inicial. Esto demuestra empíricamente que, para dimensiones moderadas, las operaciones directas sin memoria dinámica adicional superan a los enfoques de dividir y conquistar puramente teóricos. No obstante, para comprender el comportamiento a gran escala, es necesario aplicar la regla del límite para comparar funciones matemáticas. Esta regla establece que una función con menor orden de crecimiento siempre dominará el tiempo de ejecución a medida que el tamaño de entrada tienda al infinito. Por consiguiente, existirá un umbral donde las dimensiones de las matrices serán lo suficientemente masivas como para que la reducción de multiplicaciones logre compensar las constantes operativas de Strassen, momento en el cual este superará inevitablemente la velocidad del método Naive.

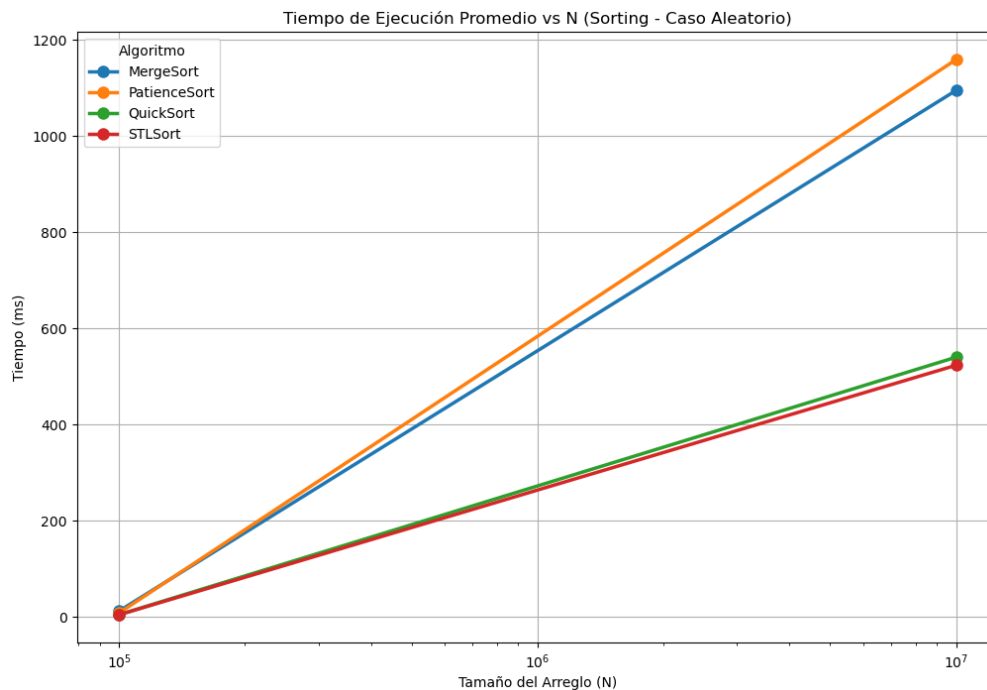


Figura 2: Tiempo de ejecución vs Tamaño del arreglo para algoritmos de ordenamiento.

3.2. Ordenamiento: Optimización de QuickSort (Antes vs Después)

•**Antes del ajuste:** La implementación original seleccionaba un pivote en el extremo del arreglo, lo que generaba un árbol de recursión completamente desbalanceado al enfrentar datos ascendentes o descendentes. Este peor caso obligaba al algoritmo a ejecutar un número de operaciones cercano a $O(n^2)$, volviéndolo tan ineficiente en la práctica como BubbleSort o SelectionSort y provocando tiempos de ejecución excesivos.

•**Después del ajuste:** Al modificar la selección del pivote hacia el centro del subarreglo, permite generar un árbol de recursión balanceado. Esta simple optimización ayuda a que se vuelva a un rendimiento promedio del algoritmo a $\Theta(n \log n)$.

Tras la corrección, QuickSort logró ordenar los 10.000.000 de elementos en fracciones de segundo, posicionándose por debajo de las curvas de tiempo respecto a los otros algoritmos.

3.3. Ordenamiento: Optimización de PatienceSort (Antes vs Después)

Antes del ajuste: El diseño inicial dependía de búsquedas lineales y eliminaciones directas en el medio de arreglos dinámicos. Eliminar elementos ubicados entre los extremos de una estructura lineal es sumamente ineficiente, requiriendo un tiempo de $O(n)$ en el peor de los casos para desplazar la memoria restante. La repetición de estos desplazamientos degradaba todo el proceso a un tiempo cuadrático $O(n^2)$.

Después del ajuste: Se implementó una cola de prioridad basada en la estructura de un Min-Heap (priority queue). La teoría respalda que un Heap permite extraer el elemento de máxima prioridad (el

mínimo) en un tiempo óptimo de $O(1)$ e insertar nuevos datos reorganizando el árbol en tiempo $O(\log n)$.

Reemplazar la búsqueda lineal por búsqueda binaria (lower bound) redujo el costo de inserción, estabilizando el tiempo global de la ejecución a un comportamiento predecible y eficiente de $O(n \log n)$.

3.4. QuickSort vs MergeSort: Sobrecarga en la mezcla

A pesar de que ambos algoritmos comparten una complejidad eficiente, la gráfica de resultados ubica a MergeSort con tiempos absolutos mayores a QuickSort para $N = 10^7$.

Esta brecha ocurre porque MergeSort requiere exactamente $2n$ asignaciones adicionales de memoria en cada nivel de recursión simplemente para traspasar los elementos hacia y desde el arreglo temporal de mezcla.

QuickSort, siendo óptimo en su caso promedio, opera directamente sobre el arreglo (in-place) mediante intercambios aislados, ejecutando una cantidad significativamente menor de asignaciones físicas en memoria.

Algoritmo (Multiplicación)	Dimensión (N)	Tiempo Promedio (ms)	Memoria (KB)
Naive	2^8	[Ingresar ms]	[Ingresar KB]
Strassen	2^8	[Ingresar ms]	[Ingresar KB]

Algoritmo (Ordenamiento)	Dimensión (N)	Tiempo Promedio (ms)	Memoria (KB)
MergeSort	10^7	[Ingresar ms]	[Ingresar KB]
QuickSort	10^7	[Ingresar ms]	[Ingresar KB]
PatienceSort	10^7	[Ingresar ms]	[Ingresar KB]
STLSort	10^7	[Ingresar ms]	[Ingresar KB]

Cuadro 1: Resumen empírico de rendimiento para las dimensiones máximas exigidas ($N = 2^8$ y $N = 10^7$).

4. Conclusiones

4.1. Respecto a algoritmos de Multiplicacion de Matrices

El análisis experimental demuestra que una complejidad asintótica teórica superior no garantiza velocidad inmediata en la práctica debido al impacto de las constantes ocultas. En la multiplicación de matrices, la sobrecarga recursiva de Strassen satura el procesador tempranamente, permitiendo que las operaciones directas del método Naive sean más veloces para la dimensión máxima exigida de $N = 2^8$. No obstante, la regla del límite asintótico establece que Strassen dominará inevitablemente a medida que el tamaño de entrada tienda al infinito.

4.2. Respecto a algoritmos de Ordenamiento

Por su parte, en los algoritmos de ordenamiento, el rendimiento depende críticamente de la gestión de memoria física y la mitigación de peores casos. QuickSort supera empíricamente a MergeSort porque

opera in-place, evitando las costosas $2n$ asignaciones adicionales de memoria por nivel de recursión que MergeSort exige para mover datos. Además, reestructurar la lógica de los algoritmos previene su colapso computacional: centrar el pivote en QuickSort fuerza un árbol balanceado que evita su degradación a $O(n^2)$, mientras que la integración de un Min-Heap en PatienceSort garantiza la extracción del mínimo en un tiempo óptimo de $O(1)$. Finalmente, la inclusión del algoritmo SORT de la librería estándar de C++ sirvió como el punto de referencia definitivo dentro de las mediciones experimentales, demostrando empíricamente la brecha de rendimiento que existe entre una implementación estandarizada y los algoritmos contruidos manualmente.